COMP101
Introduction to Programming
2025-26

Week-02 Tutorial-02

Algorithms and Languages

Compilers and Interpreters

Reading only

Algorithms

- Computer science studies the solving of problems
- Solving of problems is described using an algorithm
- An algorithm is written without thinking of any specific programming language to describe the problem – it uses structured approaches
  - Structured approaches can use two main paradigms
    - ‘Divide and Conquer’ - Pseudocode, Structured English, Flowcharts, Decision tables etc
    - ‘Top-down decomposition’ and ’top-down refinement’
- A problem is often solvable via different algorithms
  - Try to use the best one that solves the problem to deliver the result - ideally finite and effective (i.e. it works and gives the correct answer)

Algorithms

- Algorithms can be classified
  - Well-defined algorithm - steps are precise and understandable
  - Finite algorithm - must eventually solve the problem
  - Effective algorithm - solves all cases in the desired output
  - Flexible algorithm - handles a wide range of data inputs
- When we are happy with an algorithm, it can be encoded in any programming language for execution by a computer
  - we ‘write a program’ (‘code a program’) to ‘run’ (‘execute’) on a computer
  - this ‘code’ describes a solution in terms of the data, using a step-by-step list of instructions (in procedural paradigms) to represent the algorithm as a program in a computer language

COMP101

Algorithms

- An algorithm should help us understand the problem
- We use a series of well-defined steps to solve the problem
  - We usually start with a high level design
    - the easiest way is to start with INPUT – PROCESS – OUTPUT
    - COMP101 will use pseudocode
  - We can break this down to lower levels to code it using any of the structured approaches paradigms (Pseudocode, Structured English, Flowcharts, Decision tables etc)
- We could consider
    - Look for similar solutions already applied to a similar problem
    - Eliminate non-workable solutions

Language

- An algorithm is represented in a programming language by using ‘control constructs’, ‘data types’ and ‘operations’
- Control Constructs
  - Sequence
  - Selection
  - Iteration
- Data types
  - Primitive types – integer, string, floating point etc
    - low-level and built-in to the language to act as building blocks
- Operations
  - Definitions of what can be done to the data, e.g. +, -, /, * etc

Language – Control Constructs

- All languages work by using combinations of three control constructs
- Sequence
  - Every line of code in the program will execute in the order in which they are written – execution is not optional – each line MUST execute for the program to work
- Selection
  - Uses ‘if statements’ (there are others forms of this). Execution of some lines is optional – the program WILL work even if some lines do not execute
- Iteration
  - Uses ‘loop statements’ (For, While, Repeat). Iterated lines of code can be executed as many times as needed – sometimes not at all. The program WILL work if these lines are iterated or not

Language – Data types

- Any standard procedural language has two basic data types
- Numeric and String
  - There can be other data types depending on the language used
  - We concentrate here only on these two basic data types

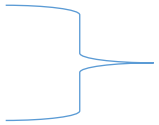
- The decision for which data type a value should be can be simplified
  - If we need to do any maths on it – it should be a numeric data type
  - If we do not need to do any maths on it – it should be a string data type

Language – Data types Numeric

- Four classes of numeric data types
  - Integers (no decimal point)
    - data type = int
    - 64-bit range = -9,223,372,036,854,775,808 to min 9,223,372,036,854,775,807
  - Floating point (have a decimal point)
    - data type = float
    - Range (min) = +/-2.2250738585072014x10^{-308} to (max) +/-1.7976931348623157x10^{308}
  - Complex (have a real part (number) and an imaginary part(number j))
  - Logical arguments
    - data type = bool (True/False, 0/1, on/off)

Language – Data types String

- Two classes of string data type
  - String
    - A group of any characters within quotes (quotes can be single or double)
      - “Cat”, ‘Cat’, etc (these are strings of length=3 because there are three characters in them)
      - “36”, etc (This is a string of length=2 because there are two characters in it
        - a number in quotes is treated as a string – no maths can be done on it
  - Char
    - A single character within quotes (quotes can be single or double)
      - “C”, “a”, “t”, etc (
      - “3”, “6”, etc



These are NOT strings of length=1, they are a ‘char’ data type because there is only one character inside the quotes

- **Python only recognises the string data type**
  - It doesn’t have the ‘char’ data type for a single character
  - Python classifies “C”, “3”, etc as **string length=1**

Language – Operations

- Operations are what we can do to the data
- Casting
  - Converts data from one type to another
  - String to numeric or numeric to string etc
  - Useful if user inputs a number as a string data type (which cannot have maths done on it) and we can convert it (‘cast it’) to a numeric data type (which can have maths done on it)
- Mathematic operations
  - For standard and advanced arithmetic etc

Language: Example
What data is to be programmed?

- Programming languages have a notation to describe the processes to be done on the data and can describe the data itself

1. to capture employee detail, assume we need the following data:

name

job

id

telephone

salary

payscale

Problem Statement:
Write a program to capture ('input') details about an employee. Input is via the keyboard

Each one of these is a 'data item'
There are six data items in total needed to solve this problem

Language:
What value do we ‘assign’ to the data

2. We need to populate each data item with a ‘value’:

name	John Smith
job	Programmer
id	1234
telephone	345-6789
salary	25000
payscale	06

This now shows each data item has an associated value.

From this point we have one further job to do – we must identify what type of data each value actually is:
i.e. what is the ‘data type’ for each value

Language: What data type is the value

3. To process the data, we need to identify the ‘data type’ of each value

name	John Smith	No maths needed → string
job	Programmer	No maths needed → string
id	1234	No maths needed → string
telephone	345-6789	No maths needed → string
salary	25000.00	May need maths → floating point
payscale	06	May need maths →integer

Language – application to the given problem

- Languages are usually developed to maximise the processing of the data, depending on the nature of the problem and the desired output
  - This is why there are so many languages and paradigms
- If struggling to make something work, we may be using the wrong language
- Sometimes a solution doesn’t work because the problem is ‘intractable’
  - It can’t be solved in a realistic amount of time

Language: Interpreter and Compiler

- ‘High level’ languages allow us to write code with human-readable statements (‘source code’)
- Source code must be translated into machine code before a computer can execute it
- This is done by an interpreter (interpreted languages include Python, Java, Ruby) or a compiler (compiled languages include C, C++, C#)
- Interpreters translate source code into ‘bytecode’ for execution by a bytecode interpreter – a ‘Virtual Machine’ (VM) which must be on the host machine to execute the code

Source Code → Bytecode → VM → CPU

Language: Interpreter and Compiler

- Compilers convert source code to machine code
- All of the source code is compiled to generate ‘object code’ in binary
- The compiler then combines this object code into a single machine code file (.exe) – brings in any libraries needed etc
- This can then execute on a computer

Source Code → Machine Code → CPU

Language: Interpreter and Compiler

Interpreter

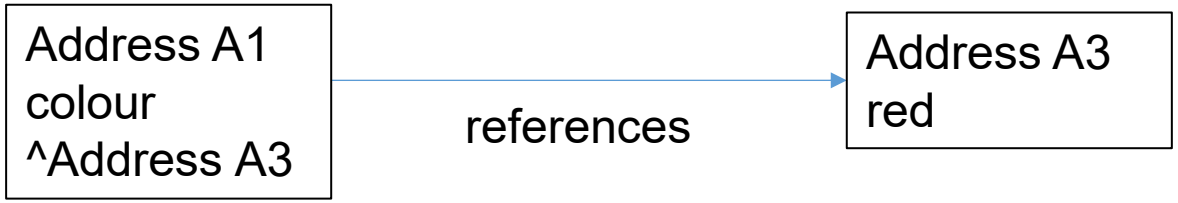
- Translates individual lines
- No intermediate object code
- Small memory requirements
- Translation done on each run
- Error report by statement
- Slower execution
  - Translates individual lines of code

Compiler

- Converts entire code
- Has intermediate object code
- Bigger memory requirements
- Conversion done once only
- Error report on entire code
- Faster execution
  - Machine code all on the computer

Assignment Statement Mechanics

- colour = input(“Enter a colour: “) red



Memory address (A1) holds the var name and a pointer (^) or ‘reference’ to another memory address

Memory address (A3) holds the value ‘red’ which is associated with variable ‘colour’

i) var name ‘colour’ will remain the same - store it in its own memory space (A1)
ii) The value it ‘holds’ can change (that’s why it’s a ‘variable’)
iii) Because it can change, the value needs to be held in its own memory location (A3)
iv) When we access ‘colour’ the interpreter translates ‘colour’ as memory location A1.
v) In A1, there is a pointer (^) to A3 where we find the current value associated with A1 (colour)

Assignment Statement Mechanics

1.User types in ‘red’ and it is assigned to a variable called colour. The value red is now associated with the variable ‘colour’

2. ‘colour’ is what we call it, but the interpreter recognises it as memory address A1 (or whatever address is free)

3. Memory address A1 does not hold the value red as typed in by the user – it holds a reference to another memory address where the value entered by the user that is now associated with the variable is really stored – here it is memory address A3

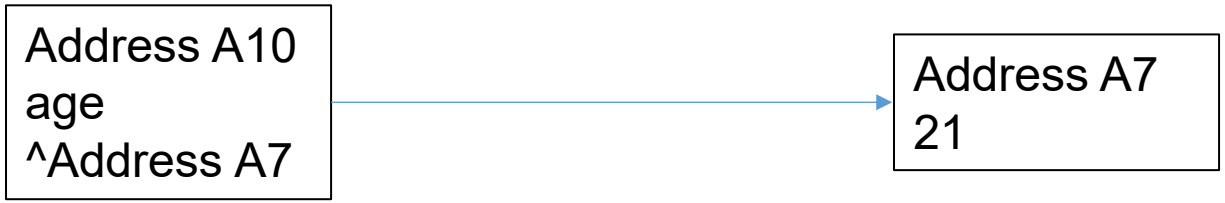
4. The address (A3) referenced by A1 is where the value red is really held, i.e., memory address A1 assigns a storage location (an ‘object reference’) which points to the value associated with the variable.

In this example, the value being pointed at is string object (’red’)

5. We don’t have memory address A1 actually holding the data ’red’

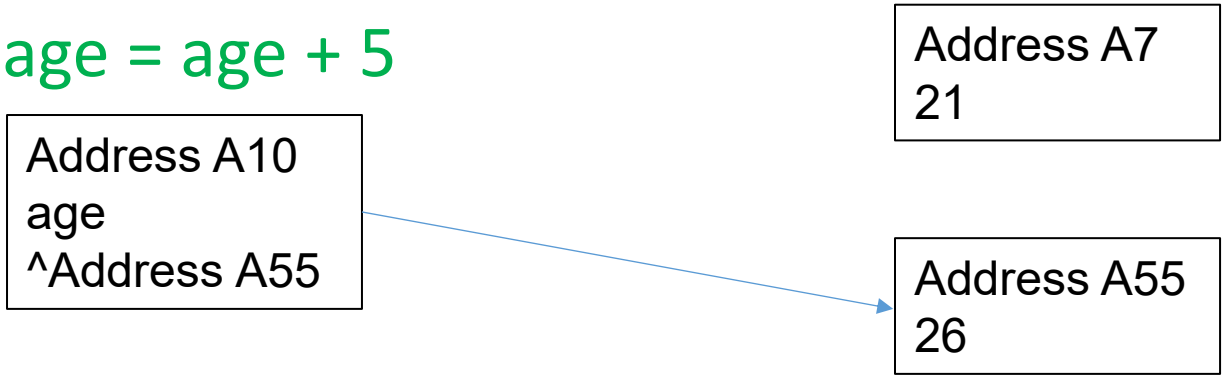
Assignment Statement Mechanics

age = 21



#add 5 to age

age = age + 5



#Address A10 (age) no longer references A7 (with original value 21);
It now references A55 (with new value for age of 26);
We have 'lost' the original value